CHECK WHAT YOU LEARNT

Check Yourself - Unit 1

A. Choose the correct answer

1. A requirement is only useful if it is
   (a) ambitious (b) testable (c) short (d) technical
2. A sensor that always reads 3 cm too far has
   (a) random error (b) systematic error (c) poor precision (d) no error
3. A sensor giving 30, 34, 27, 33 cm for one fixed object has poor
   (a) accuracy (b) precision (c) resolution (d) range
4. Random error is best reduced by
   (a) a correction factor (b) averaging several readings (c) a bigger resistor (d) recalibrating
5. 42 plus or minus 2 cm means the true value very likely lies between
   (a) 40 and 42 (b) 40 and 44 (c) 42 and 44 (d) 38 and 46
6. With 3 volts across a 220 ohm resistor, the current is about
   (a) 1.4 mA (b) 13.6 mA (c) 136 mA (d) 660 mA

B. Fill in the blanks

1 Ohm's law is written as V = _____ x _____.

2 A consistent offset that is the same every time is called _____ error.

3 The smallest change a system can detect is its _____.

4 A calibration certificate states how _____ an instrument is allowed to be.

C. Answer in one or two lines

1 Rewrite 'the robot should be fast' as a testable requirement.

2 Explain why a precise but inaccurate sensor is more dangerous than a scattered one.

3 Describe how you would estimate the uncertainty of a measurement using only data.

4 Work out the current through a 330 ohm resistor with 3 volts across it.

UNIT 2

The Arduino IDE

Sessions 3, 4 and 5

You cannot debug what you cannot picture. This unit opens the board, opens the chip, and follows your code from the letters you type to the machine instructions that actually execute.

By the end of this unit you will be able to:

- Name the parts of the Arduino Uno board and the blocks inside the ATmega328P.
- Describe every stage between pressing Upload and the program running.
- Explain what the ADC measures and calculate its resolution.
- Write non-blocking code using millis() instead of delay().

SESSION 3 — one class

Inside the Board and Inside the Chip

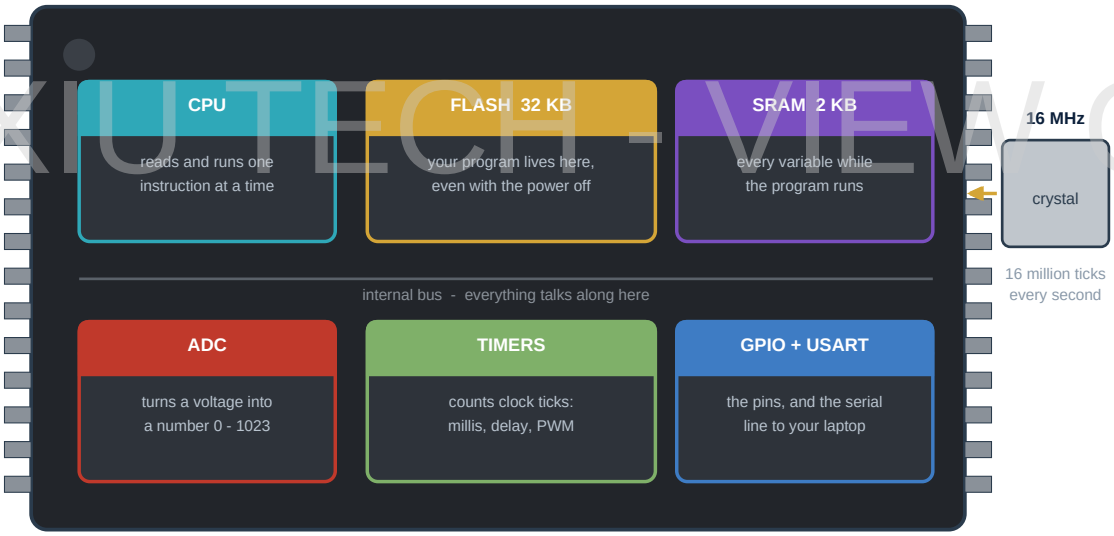
Today you will

- Identify every major component on the Arduino Uno board.
- Learn what the ATmega328P contains and how much of each.
- Confirm the whole toolchain works with a test upload.

The blue board is not the computer. It is a **support system** for one chip, and every other part on that PCB exists to make the chip convenient for a human to use.

Component on the board	Its job
ATmega328P - the long black chip	The microcontroller. This is the actual computer.
16 MHz quartz crystal	Sets the clock: sixteen million ticks every second.
Second, smaller chip near USB	Converts USB signalling into the plain serial the ATmega understands.
Three-legged regulator near the jack	Turns 7 to 12V from a battery into a steady 5V.
Electrolytic capacitors	Smooth out dips in the supply when something draws a sudden gulp.
Headers	The pins D0 to D13 and A0 to A5 you plug into.
Reset button	Restarts the program from the first instruction.

ATmega328P - the chip inside the Arduino Uno



Everything you write ends up as instructions the CPU fetches from flash, one at a time, sixteen million times a second.

Figure 2.1 - Behind the blue board: what is actually doing the work

Block inside the chip	Size	What it does	Lost on power off?
Flash	32 KB	Holds your compiled program	No
SRAM	2 KB	Holds every variable while running	Yes
EEPROM	1 KB	Small data you want kept	No
CPU	8-bit	Executes one instruction at a time	-
ADC	10-bit	Converts a voltage to 0 - 1023	-
Timers	three	Drive millis(), delay() and PWM	-

Behind the Scenes

Two kilobytes of SRAM is a genuine constraint, not a curiosity. Every variable, every array and every text message lives there simultaneously. Your maze solver will store its path in an array - and if you make that array too large, the program will not warn you, it will simply start behaving inexplicably as the memory overflows into itself. Budget your SRAM the way you would budget money.

SESSION 4

one class

From Typed Text to Running Machine Code

Today you will

- Follow the four stages of compiling and uploading.
- Understand what the bootloader is and why the board resets.
- Diagnose an error message by the stage it came from.

Between pressing Upload and your program running, four distinct things happen in about two seconds. Knowing which stage a failure came from turns a baffling red message into a two-second diagnosis.

1. **Compiling.** Your C++ text is translated into ATmega machine instructions. Any syntax error stops everything here and nothing is transmitted.
2. **Linking.** The libraries you included are combined with your code into a single file of hexadecimal machine code - the .hex file.
3. **Resetting.** The IDE pulses the board's reset line. The chip restarts and runs a small resident program called the **bootloader** for about one second.
4. **Writing.** The bootloader listens on the serial line, receives the .hex file in packets, and writes them into flash. It then jumps to your program, which begins immediately.

Behind the Scenes

The bootloader occupies about half a kilobyte of your flash and costs you a second of start-up on every reset. Professional products remove it entirely and program the chip directly through its ISP header, which is why a commercial device boots instantly. What the bootloader buys you is the ability to program the board with nothing more than a USB cable - a very good trade for a laboratory.

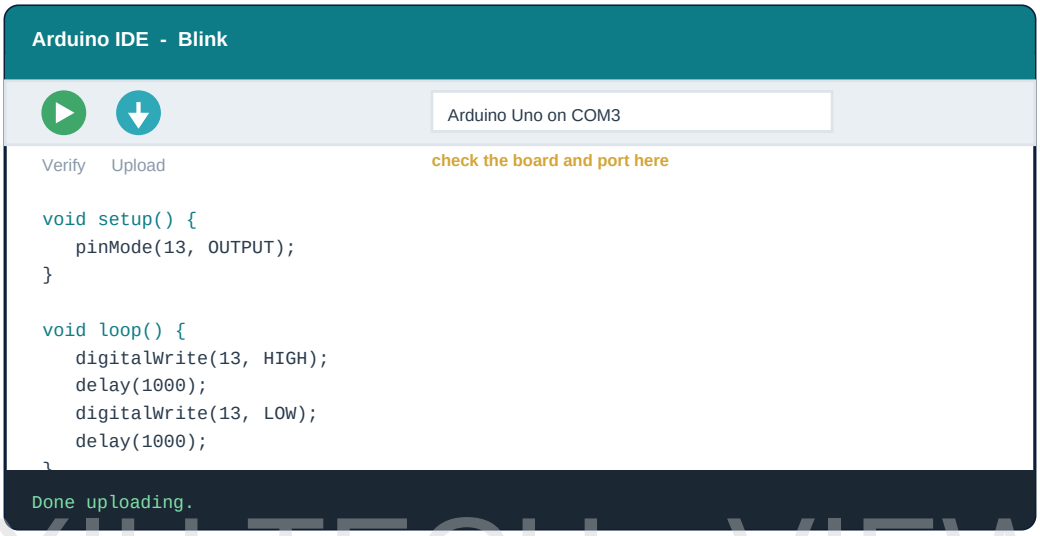


Figure 2.2 - Verify checks your code; Upload sends it to the board

Message	Stage that failed	Diagnosis
expected ';' before ...	Compiling	Missing semicolon on the line above
was not declared in this scope	Compiling	Typo, wrong capitals, or a missing #include
no matching function for call	Compiling	Wrong number or type of arguments
Sketch too big	Linking	Program exceeds 32 KB of flash
Low memory available	Linking	Variables and strings exceed comfortable SRAM
Board at COM__ is not available	Resetting	Wrong port, or a charge-only USB cable
programmer is not responding	Writing	Wrong board selected, or the bootloader was interrupted

SESSION 5

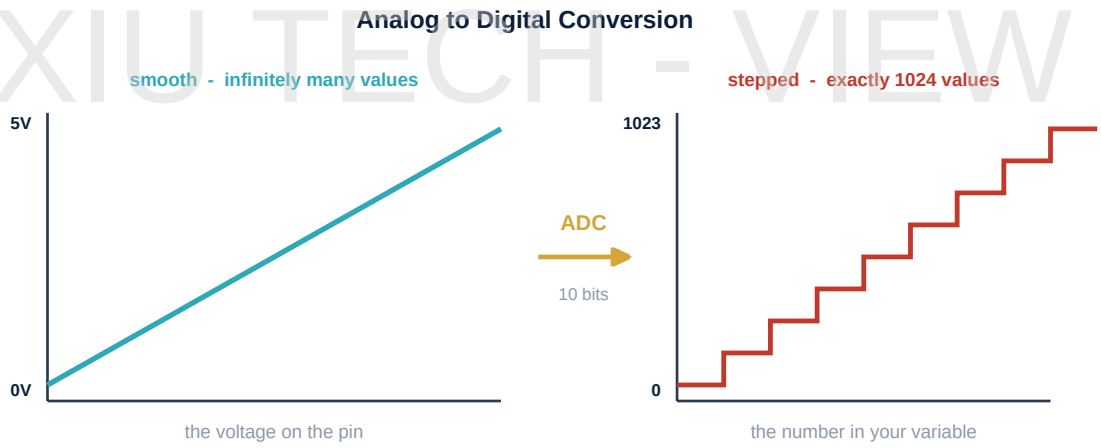
one class

Data Acquisition and Non-Blocking Code

Today you will

- Understand what the ADC measures and calculate its resolution.
- Learn why delay() is unacceptable in a real system.
- Write a non-blocking loop using millis().

An analog pin measures exactly one quantity: **voltage**. Everything else - light, distance, temperature - reaches the Arduino only after some circuit has converted it into a voltage first. Inside the chip, the Analog to Digital Converter compares that voltage against a reference and reports a whole number.



one step = 5V / 1024 = about 4.9 millivolts

a change smaller than that is invisible to the Arduino. That is its resolution.

Figure 2.2 - Every analogRead() is a measurement rounded to the nearest step

The converter is **10-bit**: 2 to the power 10, giving 1024 distinct values from 0 to 1023. Across a 5 volt reference each step is 5 divided by 1024, about 4.9 millivolts. That is the resolution, and no amount of clever code can see a change smaller than one step.

Behind the Scenes

The ADC uses **successive approximation**. It sets an internal reference to half of full scale and asks whether the input is above it, then halves the remaining range and asks again, ten times over - a binary search executed in hardware. Each conversion takes roughly 100 microseconds. In a loop polling several sensors that is not free, and in Unit 4 you will care about it.

5.1 Why delay() Is Not Acceptable

`delay(1000)` does not schedule anything. It halts the processor completely for a second. During that second your machine cannot read a sensor, cannot notice a button, cannot update a display and cannot stop a motor. In a toy that is a nuisance. In a security system or a moving robot it is a defect.

The alternative is to stop thinking in terms of *waiting* and start thinking in terms of *elapsed time*. `millis()` reports how many milliseconds the board has been running. Record the time an event happened, and on every pass of the loop ask whether enough time has gone by. The loop never stops turning.

The non-blocking pattern you will use all year

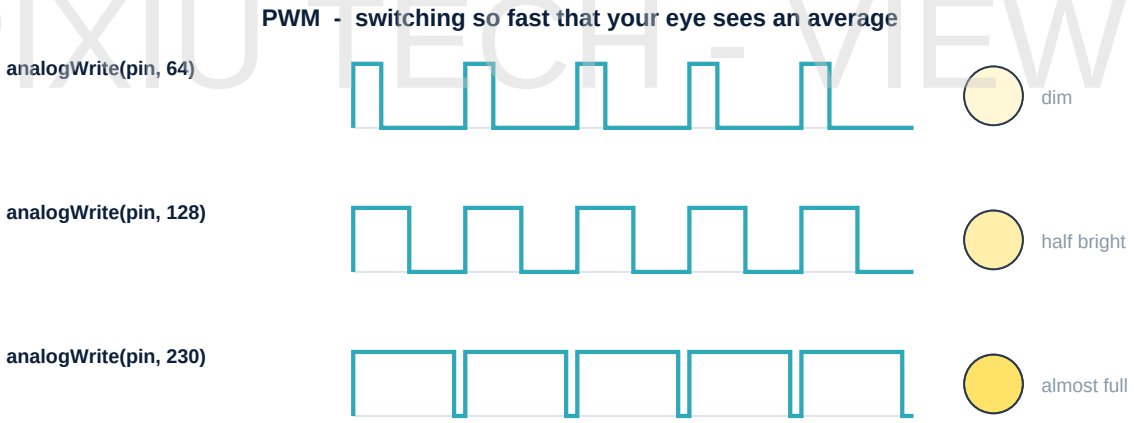
```
unsigned long lastBeep = 0;
unsigned long lastRead = 0;

void loop() {
  unsigned long now = millis();

  if (now - lastRead > 100) {     // sample the sensor ten times a second
    lastRead = now;
    // ... take a reading here ...
  }

  if (now - lastBeep > 500) {     // and blink the alarm twice a second
    lastBeep = now;
    // ... toggle the buzzer here ...
  }

  // the loop keeps running - nothing is ever frozen
}
```


Watch Out

Always declare a millis() variable as `unsigned long`. An ordinary int overflows after about 32 seconds and an ordinary long after 24 days; unsigned long lasts roughly 49 days. Writing `now - lastRead` rather than `now > lastRead + 100` also keeps working correctly even at the moment the counter wraps around.

5.2 The Reverse Operation: PWM

If the ADC turns a voltage into a number, PWM does close to the opposite. A digital pin can only be fully on or fully off, so the chip switches it about 490 times a second and varies how much of each cycle is spent on. The **average** voltage changes even though the pin never sits anywhere in between.



The pin is still only ON or OFF. It is the amount of time spent ON that changes.
Only pins marked with ~ can do this: D3, D5, D6, D9, D10 and D11.

Only the six pins driven by the timers can do this - D3, D5, D6, D9, D10 and D11, all marked with a small ~. In your capstone, `analogWrite(ena, 120)` means *hold the enable pin on for about 47 percent of each cycle*, and the motor's own inertia smooths the rest.

New word	What it means
ATmega328P	The microcontroller that is the real computer on the board.
Bootloader	A resident program that receives your code and writes it to flash.
ADC	The circuit converting a voltage to a number.
Successive approximation	The binary-search method the ADC uses.
Resolution	The smallest change a measuring system can detect.
Non-blocking	Code that waits without halting everything else.